

Apex Style Guide

Contents

1. [Naming Conventions](#)
 - Classes
 - Interfaces
 - Variables
 - Properties
 - Methods
 - Files
2. [Formatting](#)
 - Methods
 - Code Blocks
 - Classes
 - SOQL & SOSL
3. [Comments](#)
4. [Exceptions](#)

Naming

Appropriate naming helps us glean useful information quickly. Conventions facilitate self-documenting code that improves readability and consistency.

Look at the example below and try to determine the purpose of the method.

```
public List<List<Integer>> getThem() {  
    List<List<Integer>> list1 = new List<List<Integer>>();  
    for (List<Integer> x : theList) {  
        if (x.get(0) == 4) {  
            list1.add(x);  
        }  
    }  
    return list1;  
}
```

What were you able to infer about the purpose of the code above? Now look at the same example that follows naming conventions

```
public List<List<Integer>> getFlaggedCells() {  
    List<List<Integer>> flaggedCells = new List<List<Integer>>();  
    for (List<Integer> cell : gameBoard) {  
        if (cell[STATUS_VALUE] == FLAGGED) {  
            flaggedCells.add(cell);  
        }  
    }  
}
```

```
        return flaggedCells;  
    }
```

The method name of the second example not only allows us to quickly determine the purpose of the method, but we can also guess what kind of application it belongs to (looks like minesweeper to me).

[Back to Contents](#)

General Naming Rules

Use Intention-Revealing Names

- Naming is hard, but taking the time to choose a name that describes exactly what you are naming saves time for you and all others to come in the future
- The name should speak for itself. If the name requires a comment to further explain the intent, the name needs to be revised
- Do not be afraid of renaming if you think of a better, more descriptive one in the future (modern editors make this renaming a trivial task)

Avoid Disinformation

Make Meaningful Distinctions

Distinguish names so that the reader knows what the differences are. For example, the class names `PersonInfo` and `PersonData` have different names, but the names don't *mean* anything different

Use Pronounceable Names

We write code for humans to read, so make sure humans can discuss it with our spoken language.

```
// BAD  
private DateTime lastChangeYMDHMS;  
// GOOD  
private DateTime lastChangeDateTime
```

Use Searchable Names

- Names should be easy to locate across a body of text.
- The length of the name should correspond to the size of its scope.

Avoid Encodings

Encoding type or scope information into names only adds to the burden of deciphering intent. They are a mental burden, rarely pronounceable, and easy to mistype

Don't Be Cute

Don't try to be clever or funny with your names. The humor will disappear very quickly when somebody is trying to quickly decipher your code in order to fix an issue.

Pick One Word per Concept

Pick and use one word for an abstract concept and stick with it. For example, what is the difference between *fetch*, *retrieve*, and *get*?

Don't Add Gratuitous Context

- Shorter names are generally better than longer ones, as long as they are clear
- Add no more to the name than is necessary

[Back to Contents](#)

Naming Classes

- Class names should be in pascal case
 - They should start with a capitalized letter
 - The first letter of all subsequent words should be capitalized
- Should have noun or noun phrase names like User, Customer, AddressParser
- DO NOT use words like Manager, Processor, Data or Info, they do not add anything to describing the purpose of the class
- Avoid using design pattern names like Wrapper. A wrapper class could be called SelectableAccount instead of AccountWrapper
- Only use alpha characters

Controller classes should end with the word *Controller*

Web Service classes should end with the word *Service*

Aura enabled Controller classes should end with *AuraService*

Trigger classes should be named the object they are triggers for, and should end with *Trigger*

Test classes should be named the same as the class they are testing, except with *Tests* at the end

Naming Interfaces

- Follow the same rules as classes, except put *Interface* at the end

Naming Variables

You should name a variable using the same care with which you name a first-born child - *Robert C. Martin*

- Variables are named using camel case
- Avoid abbreviations (other than common ones like Id, app, org, etc...)

- Common acronyms can be used (ssn, dob, DB)
- Variable types in the name are not necessary (don't do `stringAlertMessage`, or `intErrorCount`)
- Booleans should be named to let you know that it is a boolean (`hasErrors`, `isFull`, etc...)
- The name should be long enough to describe its intent
 - The only exception to this is a variable in a very small scope, so in a small for loop for example
- Distinguish between single objects and collections (`selectedAccount` vs `selectedAccounts`)
- Constants get named in all uppercase with an `_` between each word `SOME_CONSTANT`

Naming Properties

- Properties are named using pascal case
 - They should start with a capitalized letter
 - The first letter of all subsequent words should be capitalized
- Follow the guidelines in naming variables for the other rules

```
public String AccountName {get; set;}
```

Naming Methods

- Method names should be in camel case
 - They should start with a lower-case letter
 - The first letter of all subsequent words should be capitalized
- Method names should have a verb and follow one of the patterns below
 - verb-Noun or verb-Noun-How (`getAccountsFromId`, `getUsers`)
 - Verb-adj-noun (`getSelectedUsers`, `hasSelectedItems`)
 - Ver-adj (`isDeleted`) Boolean returns always use present tense states of being: `is`, `has`, `are`, etc
- Accessors, mutators, and predicates should be named for their value and prefixed with `get`, `set`, and `is`
- Test methods should follow this syntax: `methodUnderTest_testCondition_expectedResult`

```
// Method under test
function createIndividualizedGreeting(name) {
    return 'Hello, ' + name;
}

// Test method
function
createIndividualizedGreeting_givenName_shouldAppendNameToTheEndOfGreeting(
) {
    String greeting = createIndividualizedGreeting('Gob');
    System.assertEquals('Hello, Gob', greeting);
}
```

Naming Visualforce Pages & Components

- Follow the same rules as classes, so `CustomerAccountDetails`, or `SearchAccounts`

- Follow the rules for variables above when naming apex tags and their attributes

Naming Files

- for the most part, the file name should reflect the name of your class
- prefix file names to make sure that related files show up next to each other since Salesforce does not give us foldering

[Back to Contents](#)

Formatting

Braces

Opening Braces

- The opening brace "{" for the classes/interfaces, methods or any other code blocks (if/else, while, for, try/catch, etc) should begin at the end of the same line as the construct
- Leave a space before the opening brace
- Closing braces should start on a new line indented to match the corresponding opening statement

```
function showWhereBracesGo() {  
    // some stuff  
}
```

Braces for Property getter and setter methods

- Property declarations with default get and set methods should include opening and closing braces on the same line as the declaration

```
public String Name { get; set; }
```

- Properties with more complicated getters and setters should follow the braces rules above

```
private String privateName;  
public String Name {  
    get {  
        if (someCondition) {  
            return privateName;  
        } else {  
            return 'Gob';  
        }  
    }  
    set {  
        if (someOtherCondition) {  
            privateName = value;  
        }  
    }  
}
```

```

        } else {
            privateName = 'Michael';
        }
    }
}

```

Braces for single line blocks

if, while, and for statements should always use braces, even if they have a single statement

```

// GOOD
if (someCondition) {
    return 'Gob';
}

// BAD
if (someCondition)
    return 'Gob';

```

Indentation

Indentation should use tabs, not spaces and tab size should equal 4 spaces

Line Wrapping

When an expression will not fit on a single line, break it according to these general principles:

- Start thinking about breaking to the new line once you hit around 50 characters
- Once you hve hit 80 characters in a line you have gone too far and need to break that line up
- Break before an operator
- Break after a comma
- Prefer higher-level breaks to lower-level breaks
- Indent at least once after the break

```

// GOOD
someReallyReallyLongName = someOtherLongVariableName
                                * (someLongName +
someOtherVariable - thirdLongVariable)
                                + 4 *
someBigMultiplier;

// BAD
// Too Long
someReallyReallyLongName = someOtherLongVariableName * (someLongName +
someOtherVariable - thirdLongVariable) + 4 * someBigMultiplier;

// No Indent

```

```
someReallyReallyLongName = someOtherLongVariableName * (someLongName +
someOtherVariable - thirdLongVariable) + 4 * someBigMultiplier;

// Breaks inside the parens, too low level
// Also broke after the + operator
someReallyReallyLongName = someOtherLongVariableName * (someLongName +
    someOtherVariable - thirdLongVariable) + 4 *
someBigMultiplier;
```

Formatting Methods

Methods should start have no space between the method name and the params (and then one space between the ending) and the beginning {

```
function updateContact(Contact contactToUpdate) {
    ...
}
```

One thing to note about methods is that they should be kept as small as they need to be. Methods should have only 1 responsibility, so if you start to see your method going over around 5 lines of logic start to prepare for some refactoring. Methods should also accept no more than 3 arguments. Using more than 3 arguments is an indication that your method is doing too much. Methods should avoid using Booleans as arguments. It is much better to evaluate the boolean and call two different methods based on the boolean value. This enforces single responsibility principles.

Formatting Code Blocks

The only real rule for formatting code blocks is to try to keep them short, and only 1 level of indentation. Having deep nesting in your code blocks makes it difficult to read, it also screams that your code block/method is doing a little too much. Split that deeper nested logic out into some well-named private methods.

```
// GOOD
for(Integer i = 0; i < 20; i++) {
    if (i > 10) {
        handleLargerThanTen();
    } else {
        handleLessThanTen();
    }
}

// AVOID
for(Integer i = 0; i < 20; i++) {
    if (i > 10) {
        if (i % 2 == 0) {
            // do something w/ even #s
        } else {
            // do something else
        }
    }
}
```

```

    }
    } else {
        if (i % 3 == 0) {
            // do some stuff
        } else {
            do some other stuff
        }
    }
}

```

Formatting Queries

SOQL

Your SOQL queries should adhere to the following rules:

- Keywords are all capital letters, **SELECT**, **FROM**, **WHERE**
- One field per line with the comma after the field, first field goes after the initial **SELECT**, all other fields go on the new line indented to start of the first field
- Any **AND** keywords get an indent
- Sub-selects get indented one level

```

// Good, easy to read, easy to modify
List<Person__c> peopleNamedGob = [SELECT Id
                                   ,
                                   FirstName__c
                                   ,
                                   LastName__c
                                   ,
                                   FavoriteColor__c
                                   FROM
                                   Person__c
                                   WHERE
                                   FirstName__c = 'Gob'
                                   AND LastName__c = 'Bluth'];

// This is very hard to read, and too long, nobody wants to have to scroll
to read the entire query
List<Person__c> peopleNamedGob = [SELECT Id, FirstName__c, LastName__c,
FavoriteColor__c FROM Person__c WHERE FirstName__c = 'Gob' AND LastName__c
= 'Bluth'];

```

[Back to Contents](#)

Comments

Every time you write a comment, you should grimace and feel the failure of your ability of expression -
Robert C. Martin

Comments should be used to explain things the code cannot explain on its own, such as the reasoning behind complex business logic. Comments should not describe what your code is doing, if you need to leave a comment explaining what your code is doing, you need to refactor your code to make it more readable.

Code does not lie, comments do. When code changes, but the comments do not, the comment is now a lie. This happens all the time. Don't be a purveyor of lies.

Methods should be named well enough to not need comments. If you need a comment saying what your method is doing, you either have a name that isn't descriptive enough, or your method is doing too much.

Code Style

Triggers

Bulkify queries

Always bulkify queries in triggers to avoid hitting governor limits and improve performance.

Use a facade

Use the [facade pattern](#) to simplify your triggers. Do not put any actual business logic inside the trigger, extract that out into another facade class.

Prevent recursion

Take appropriate measures to prevent unwanted recursive execution of triggers. For example, updating a record in the **after update** trigger of the same object or a related object could invoke the trigger recursively. Use **before update** if possible or use a static flag variable to prevent recursion. When you use **after update** to make changes on the same record, you need to use **update** statement to persist the changes, and that results in firing before/after triggers again. You can avoid that by using **before update** triggers as you wouldn't then need to use the update statement. However, certain situations require the use of **after update** in which case you should use flag variables to prevent recursion.

Single trigger for one object

Only create 1 trigger per object.

[Back to Contents](#)

Apex Classes

Future methods

Avoid writing logic directly in future methods. Instead, write the logic in a separate method so that it can be used in both the future and immediate method.

```
@future
public void updateAccountFuture(Id accountId) {
    updateAccount(accountId);
}
```

```
}

public void updateAccount(Id accountId) {
    ...
}
```

Group your methods

Group related methods together. This makes the code easier to navigate, and also helps to point out where you may have a new class hiding in your existing class.

Do not use asserts in main code

Asserts are for tests, not main code.

If statements

Complicated if conditions should go in their own well-named method.

Below is a distracting, difficult to understand if condition.

```
if (userAccount.IsActive__c && !userAccount.IsInArrears__c &&
    userAccount.Contact.IsCurrent__c) {
    // do some stuff
}
```

Instead, abstract that condition out into a method whose name says what that condition is checking

```
function isAccountCurrent(Account userAccount) {
    return userAccount.IsActive__c && !userAccount.IsInArrears__c &&
        userAccount.Contact.IsCurrent__c;
}

function doSomeStuff(Account userAccount) {
    if (isAccountCurrent(userAccount)) {
        // do some stuff
    }
}
```

Nested if statements should go into their own method.

[Back to Contents](#)

SOQL Queries

- Avoid putting SOQL queries in any sort of loop (for, forEach, while, etc)

- The database access is a responsibility, so it belongs in its own class. Your method should only have 1 responsibility, database access is not one of them

```
// Avoid
function manipulateAccount(Id accountId) {
    Account userAccount = [SELECT Id, ..., FROM Account WHERE Id =
:accountId];

    // some if statements and a little object manipulation stuff

    update userAccount;
}
```

So, what happens if we have to select account by its Id multiple times? We are going to end up with that same SOQL query scattered throughout our code.

What happens when we add a new field to that object? We will have to find all of those SOQL queries and update them to return the new field.

If we just centralized that query into its own data accessor class and method, it would be reused throughout our code, and we would only have to make that field update in one place.

Put the above SOQL query inside a class named **AccountDataAccessor**, inside a method called **getAccountById(Id accountId)**.

[Back to Contents](#)

Exceptions

Exceptions should always be handled. You need to catch exceptions at minimum at the highest level of your code before the user gets to it (so usually the controller level).

You should not throw standard exceptions up to the user. Create a custom exception with a user-friendly error message in there.

Exceptions for visualforce controllers should be caught, then added to the PageMessages.

Exceptions for AuraServices should return an AuraHandledException with a friendly error message.

[Back to Contents](#)